



SPRING DATA



Guía Paso a Paso – Creación de la clase entidad `Persona` y limpieza del controlador



Introducción

En esta lección aprenderás a:

- ✓ Convertir la clase `Persona` en una **entidad JPA**
- ✓ Preparar el proyecto para conectarse a una base de datos real usando Spring Data



Paso 1: Limpiar el controlador

Abre la clase:

```
src/main/java/gm/InicioControlador.java
```



Cámbiala al nuevo paquete `gm.web` (clic derecho → Refactor → Move) para mantener mejor organización.



Elimina todas las líneas que hacen referencia a variables como `mensaje`, `saludo` y la lista de personas creadas manualmente.

Deja solo el método `inicio()` con el `Model` y el log, así:

```
package gm.web;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ControladorInicio {

    private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

    @GetMapping("/")
    public String inicio(Model model) {
        log.info("Ejecutando el controlador Spring MVC");
        //model.addAttribute("personas", personas);
        return "index";
    }
}
```

💡 Deja comentada la línea de `model.addAttribute("personas", personas)`; ya que se usará más adelante al recuperar los datos desde la base de datos.

Paso 2: Convertir `Persona` en una entidad JPA

Abre la clase:

```
src/main/java/gm/domain/Persona.java
```

▼ Agrega las siguientes anotaciones e imports:

```
package gm.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.io.Serializable;

@Entity
@Table(name="persona")
public class Persona implements Serializable{
```

```
private static final long serialVersionUID = 1L;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private String nombre;
private String apellido;
private String email;
private String telefono;

public Persona() {
}

public Persona(String nombre, String apellido, String email, String telefono) {
    this.nombre = nombre;
    this.apellido = apellido;
    this.email = email;
    this.telefono = telefono;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
```

```

        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    @Override
    public String toString() {
        return "Persona{" + "nombre=" + nombre + ", apellido=" + apellido + ", email=" + email + ",
telefono=" + telefono + '}';
    }
}

```

Explicación:

- `@Entity`: marca la clase como entidad JPA
- `@Table(name = "persona")`: asegura que se use el nombre exacto de la tabla (todo en minúsculas), útil especialmente en sistemas como Linux o macOS donde sí importa la diferencia
- `@Id`: indica el campo clave primaria
- `@GeneratedValue`: define cómo se genera automáticamente la clave (estrategia `IDENTITY`)
- `Serializable` y `serialVersionUID`: aseguran compatibilidad al persistir objetos

Paso 3: Verificar que la configuración funciona

Haz clic derecho sobre el proyecto → Clean and Build

Si todo está bien, en la consola verás el siguiente mensaje de Hibernate:

```
HHH000204: Processing PersistenceUnitInfo [name: default]
```

✅ Esto indica que **Spring Boot creó automáticamente la unidad de persistencia default**. No necesitas crear `persistence.xml`.

Conclusión

Con esta lección has aprendido a:

- ✅ Organizar el código moviendo el controlador a un paquete `web`
- ✅ Eliminar datos estáticos para trabajar con una base real
- ✅ Declarar tu primera entidad con JPA e Hibernate

→ En la siguiente lección crearemos la **interfaz DAO** que nos permitirá recuperar los datos desde MySQL usando **Spring Data JPA**.

Nota: Aún no deben ejecutar el proyecto, ya que haremos varios cambios más antes de poder ejecutarlo y desplegar los datos obtenidos de la base de datos. Esto lo terminaremos en las siguientes lecciones.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)